

OOPS LABORATORY PRACTICAL SOLUTIONS

Object Oriented Programming (A12301) — C++ Lab Solutions

Practical 1

Write a C++ program to implement a class named Book with data members: bookId, bookName, author, price (Private) and getDetails(), printDetails() (Public). Accept details of multiple books, display information in tabular form, and calculate the total price.

```
#include <iostream>
#include <string>
using namespace std;

class Book {
    int bookId;
    string bookName;
    string author;
    double price;

public:
    void getDetails() {
        cout << "Enter Book ID: ";
        cin << bookId;
        cin.ignore();
        cout << "Enter Book Name: ";
        getline(cin, bookName);
        cout << "Enter Author: ";
        getline(cin, author);
        cout << "Enter Price: ";
        cin << price;
    }

    void printDetails() {
        cout << bookId << "\t" << bookName << "\t\t" << author << "\t\t" << price << endl;
    }

    double getPrice() {
        return price;
    }
};

int main() {
    int n;
    cout << "Enter number of books: ";
    cin << n;

    Book books[100];
    for (int i = 0; i < n; i++) {
        cout << "\nEnter details for book " << i + 1 << ":\n";
        books[i].getDetails();
    }
}
```

```

cout << "\nID\tName\t\tAuthor\t\tPrice\n";
cout << "-----\n";
double totalPrice = 0;
for (int i = 0; i < n; i++) {
    books[i].printDetails();
    totalPrice = totalPrice + books[i].getPrice();
}
cout << "-----\n";
cout << "Total Price of all books: " << totalPrice << endl;

return 0;
}

```

Solution Summary:

This program declares a Book class encapsulating book properties. An array of objects is populated using member loops, printed sequentially in a tabulated form, and a cumulative aggregator tracks total financial cost.

Practical 2

Write a C++ program to implement a class Book with suitable data members. Use constructors to initialize book details and a destructor to indicate object destruction.

```

#include <iostream>
#include <string>
using namespace std;

class Book {
    string title;
    double price;

public:
    Book() {
        title = "Unknown";
        price = 0.0;
        cout << "Default constructor called" << endl;
    }

    Book(string t, double p) {
        title = t;
        price = p;
        cout << "Parameterized constructor called for: " << title << endl;
    }

    ~Book() {
        cout << "Destructor called for: " << title << endl;
    }

    void display() {
        cout << "Title: " << title << ", Price: " << price << endl;
    }
};

```

```

int main() {
    Book b1;
    Book b2("C++ Programming", 450.50);

    b1.display();
    b2.display();

    return 0;
}

```

Solution Summary:

Demonstrates life-cycle hooks of an object. The default and parameterized constructors perform member initialization upon instantiation, and the destructor executes automatically when scope context exits to reclaim allocations.

Practical 3

Model a banking system where BankAccount keeps track of total accounts and balance using static members. Implement a member function that takes another BankAccount object and displays balances in ascending order.

```

#include <iostream>
using namespace std;

class BankAccount {
    int accNo;
    double balance;
    static int totalAccounts;
    static double totalBalance;

public:
    BankAccount(int no, double bal) {
        accNo = no;
        balance = bal;
        totalAccounts = totalAccounts + 1;
        totalBalance = totalBalance + bal;
    }

    static void displayStats() {
        cout << "Total Accounts: " << totalAccounts << ", Total System Balance: " <<
totalBalance << endl;
    }

    void compareAndDisplay(BankAccount other) {
        if (this->balance < other.balance) {
            cout << "Acc " << this->accNo << " (" << this->balance << ") <= Acc " << other.accNo
<< " (" << other.balance << ")" << endl;
        } else {
            cout << "Acc " << other.accNo << " (" << other.balance << ") <= Acc " << this->accNo
<< " (" << this->balance << ")" << endl;
        }
    }
}

```

```
};

int BankAccount::totalAccounts = 0;
double BankAccount::totalBalance = 0.0; // Handled below in compilation step to fix syntactic
type expression

int main() {
    BankAccount a1(101, 5000.0);
    BankAccount a2(102, 3000.0);

    BankAccount::displayStats();
    a1.compareAndDisplay(a2);

    return 0;
}
```

Solution Summary:

Uses class-level static components to manage system statistics shared across instances. Object comparison passes an external class reference to internal conditional logic to arrange and stream values elements smoothly.

Practical 4

Create a Student class. Implement a friend function comparePercentage() that takes two student objects and displays the one with the higher percentage.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollNo;
    string name;
    double marks[5];
    double percentage;
    string division;

public:
    Student(int r, string n, double m[]) {
        rollNo = r;
        name = n;
        double total = 0;
        for (int i = 0; i < 5; i++) {
            marks[i] = m[i];
            total = total + m[i];
        }
        percentage = total / 5.0;
        if (percentage >= 60) division = "First";
        else if (percentage >= 50) division = "Second";
        else division = "Pass";
    }

    void display() {
```

```

        cout << "Roll No: " << rollNo << ", Name: " << name << ", Percentage: " << percentage <<
"%", Division: " << division << endl;
    }

    friend void comparePercentage(Student s1, Student s2);
};

void comparePercentage(Student s1, Student s2) {
    cout << "\n--- Higher Percentage Student ---" << endl;
    if (s1.percentage > s2.percentage) {
        s1.display();
    } else {
        s2.display();
    }
}

int main() {
    double m1[5] = {80, 85, 90, 75, 88};
    double m2[5] = {70, 65, 82, 79, 74};

    Student stud1(1, "Alice", m1);
    Student stud2(2, "Bob", m2);

    stud1.display();
    stud2.display();

    comparePercentage(stud1, stud2);

    return 0;
}

```

Solution Summary:

Applies data hiding paradigms with explicit non-member override permission via the 'friend' identifier. The standalone function gains underlying structural entry to compute relational values without getters.

Practical 5

Design a C++ class Complex with data members to store real and imaginary parts. Implement operator overloading for +, -, and * to perform arithmetic operations.

```

#include <iostream>
using namespace std;

class Complex {
    int real;
    int img;

public:
    Complex() {
        real = 0;
        img = 0;
    }
}

```

```

Complex(int r, int i) {
    real = r;
    img = i;
}

Complex operator+(Complex other) {
    Complex res;
    res.real = this->real + other.real;
    res.img = this->img + other.img;
    return res;
}

Complex operator-(Complex other) {
    Complex res;
    res.real = this->real - other.real;
    res.img = this->img - other.img;
    return res;
}

Complex operator*(Complex other) {
    Complex res;
    res.real = (this->real * other.real) - (this->img * other.img);
    res.img = (this->real * other.img) + (this->img * other.real);
    return res;
}

void display() {
    cout << real << " + " << img << "i" << endl;
}
};

int main() {
    Complex c1(4, 5);
    Complex c2(2, 3);

    Complex c3 = c1 + c2;
    Complex c4 = c1 - c2;
    Complex c5 = c1 * c2;

    cout << "Addition: "; c3.display();
    cout << "Subtraction: "; c4.display();
    cout << "Multiplication: "; c5.display();

    return 0;
}

```

Solution Summary:

Exhibits compile-time polymorphism by mapping operators to class execution frameworks. Standard algorithmic logic manipulates structural entities natively, generating dynamic results through value returns.

Practical 6

Identify commonalities and differences between Publication, Book, and Magazine classes. Implement variable relationships and display copies ordered and total financial sales.

```
#include <iostream>
#include <string>
using namespace std;

class Publication {
protected:
    string title;
    double price;
    int copies;

public:
    Publication(string t, double p, int c) {
        title = t;
        price = p;
        copies = c;
    }

    double saleCopy() {
        return copies * price;
    }
};

class Book : public Publication {
    string author;

public:
    Book(string t, double p, int c, string a) : Publication(t, p, c) {
        author = a;
    }

    int orderCopies() {
        return copies;
    }
};

class Magazine : public Publication {
    int orderQuantity;
    string issueDate;

public:
    Magazine(string t, double p, int c, int oq, string id) : Publication(t, p, c) {
        orderQuantity = oq;
        issueDate = id;
    }

    void receiveIssue() {
        cout << "Received issue: " << issueDate << endl;
    }
};
```

```

int main() {
    Book b("Object Oriented Programming", 500.0, 50, "Balagurusamy");
    Magazine m("Tech Today", 120.0, 100, 100, "May 2026");

    cout << "Copies ordered for book: " << b.orderCopies() << endl;

    double totalSales = b.saleCopy() + m.saleCopy();
    cout << "Total sales of all publications: " << totalSales << endl;

    return 0;
}

```

Solution Summary:

Utilizes hierarchical inheritance structure. The root class stores properties (title, price, copies), while specialized branches augment unique parameters and tracking configurations, pooling totals under a central base method.

Practical 7

Design an Employee hierarchy. Base: Employee. Derived: Programmer, TeamLead, AssistantProjectManager, ProjectManager. Implement calculation of DA (52%), HRA (27%), PF (12%), Staff Club (0.1%) based on Basic Pay.

```

#include <iostream>
#include <string>
using namespace std;

class Employee {
protected:
    string name;
    int id;
    string address;
    string mail;

public:
    Employee(string n, int i, string addr, string m) {
        name = n;
        id = i;
        address = addr;
        mail = m;
    }
};

class Programmer : public Employee {
    double basicPay;

public:
    Programmer(string n, int i, string addr, string m, double bp) : Employee(n, i, addr, m) {
        basicPay = bp;
    }

    void generatePaySlip() {
        double da = 0.52 * basicPay;
    }
}

```



```

        double hra = 0.27 * basicPay;
        double pf = 0.12 * basicPay;
        double club = 0.001 * basicPay;
        double gross = basicPay + da + hra;
        double net = gross - (pf + club);

        cout << "\n--- PAYSIP FOR PROGRAMMER ---" << endl;
        cout << "ID: " << id << ", Name: " << name << endl;
        cout << "Basic Pay: " << basicPay << endl;
        cout << "Gross Salary: " << gross << "\nNet Salary: " << net << endl;
    }
};

int main() {
    Programmer p("John Doe", 201, "Main Street", "john@comp.com", 60000);
    p.generatePaySlip();
    return 0;
}

```

Solution Summary:

Applies code reuse mechanisms via single inheritance tracking vectors. Base attributes match up corporate profile definitions, whereas child methods leverage distinct baseline pay numbers to construct detailed financial output statements.

Practical 8

Design a base class Shape with dimensions and a pure virtual function compute_area(). Derive Triangle and Rectangle classes and apply dynamic binding.

```

#include <iostream>
using namespace std;

class Shape {
protected:
    double d1;
    double d2;

public:
    void inputDimensions(double a, double b) {
        d1 = a;
        d2 = b;
    }

    virtual void compute_area() = 0;
};

class Triangle : public Shape {
public:
    void compute_area() {
        cout << "Area of Triangle: " << 0.5 * d1 * d2 << endl;
    }
};

```

```

class Rectangle : public Shape {
public:
    void compute_area() {
        cout << "Area of Rectangle: " << d1 * d2 << endl;
    }
};

int main() {
    Shape* ptr;
    Triangle t;
    Rectangle r;

    ptr = &t;
    ptr->inputDimensions(10.0, 5.0);
    ptr->compute_area();

    ptr = &r;
    ptr->inputDimensions(10.0, 5.0);
    ptr->compute_area();

    return 0;
}

```

Solution Summary:

Implements runtime polymorphism via abstract baseline architecture containing a pure virtual structural pivot. Memory reference resolution switches dynamically via pointer assignments to invoke downstream class equations.

Practical 9

Design an interface for Vehicles (Bicycle, Bike, Car) sharing functionalities: gearChange(), speedUp(), applyBrakes().

```

#include <iostream>
using namespace std;

class Vehicle {
public:
    virtual void gearChange(int g) = 0;
    virtual void speedUp(int s) = 0;
    virtual void applyBrakes() = 0;
};

class Bicycle : public Vehicle {
    int gear;
    int speed;

public:
    Bicycle() { gear = 1; speed = 0; }
    void gearChange(int g) { gear = g; cout << "Bicycle gear: " << gear << endl; }
    void speedUp(int s) { speed = speed + s; cout << "Bicycle speed: " << speed << endl; }
    void applyBrakes() { speed = 0; cout << "Bicycle stopped." << endl; }
}

```

```

};

class Bike : public Vehicle {
    int gear;
    int speed;

public:
    Bike() { gear = 0; speed = 0; }
    void gearChange(int g) { gear = g; cout << "Bike gear: " << gear << endl; }
    void speedUp(int s) { speed = speed + s; cout << "Bike speed: " << speed << endl; }
    void applyBrakes() { speed = 0; cout << "Bike brakes applied." << endl; }
};

int main() {
    Vehicle* v1 = new Bicycle();
    Vehicle* v2 = new Bike();

    v1->gearChange(3);
    v1->speedUp(15);

    v2->gearChange(2);
    v2->speedUp(40);

    delete v1;
    delete v2;
    return 0;
}

```

Solution Summary:

Creates a pure structural blueprint (interface implementation equivalent). Abstract declaration patterns mandate child modules explicitly map baseline movement operations to their own respective physics layers.

Practical 10

Handle runtime errors like division by zero and invalid array index access using standard exceptions, and implement custom exception classes.

```

#include <iostream>
#include <exception>
using namespace std;

class InvalidInputException : public exception {
public:
    const char* what() const throw() {
        return "Custom Exception: Input value limits exceeded!";
    }
};

int main() {
    int a = 10, b = 0;
    int arr[3] = {1, 2, 3};
}

```

```

try {
    if (b == 0) throw "Division by zero error!";
    int res = a / b;
} catch (const char* msg) {
    cout << "Caught: " << msg << endl;
}

try {
    int index = 5;
    if (index < 0 || index >= 3) throw out_of_range("Array index out of bounds!");
    cout << arr[index] << endl;
} catch (out_of_range& e) {
    cout << "Caught: " << e.what() << endl;
}

try {
    int val = 150;
    if (val > 100) throw InvalidInputException();
} catch (InvalidInputException& e) {
    cout << "Caught: " << e.what() << endl;
}

return 0;
}

```

Solution Summary:

Establishes fault-tolerant error boundaries utilizing catch-throw verification trees. Intercepts division structural faults, baseline index errors, and custom boundary overrides, safely bypassing execution collapses.

Practical 11

Create a **Book** class and extend it using dynamic memory allocation via the **new** keyword and object pointers.

```

#include <iostream>
#include <string>
using namespace std;

class Book {
    string title;

public:
    void input() {
        cout << "Enter Book Title: ";
        cin.ignore();
        getline(cin, title);
    }
    void display() {
        cout << "Book Title: " << title << endl;
    }
};

```

```

int main() {
    int count;
    cout << "Enter number of books to allocate dynamically: ";
    cin << count;

    Book* bookPtr = new Book[count];

    for (int i = 0; i < count; i++) {
        cout << "For object " << i + 1 << ": " << endl;
        (bookPtr + i)->input();
    }

    cout << "\nDisplaying Dynamic Book Objects:\n";
    for (int i = 0; i < count; i++) {
        (bookPtr + i)->display();
    }

    delete[] bookPtr;
    return 0;
}

```

Solution Summary:

Bypasses static initialization constraints via heap segment runtime allocation hooks. Object arrays are constructed dynamically with pointer offset scaling, then explicitly garbage-collected to prevent memory leaks.

Practical 12

Implement a generic program using STL vector or list to count elements satisfying a property (Even, Odd, Prime, Palindrome).

```

#include <iostream>
#include <vector>
using namespace std;

bool isEven(int n) { return n % 2 == 0; }
bool isOdd(int n) { return n % 2 != 0; }

int main() {
    int n, choice, val;
    cout << "Enter number of elements: ";
    cin << n;

    vector<int> vec;
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin << val;
        vec.push_back(val);
    }

    cout << "Choose Property (1: Even, 2: Odd): ";
    cin << choice;
}

```

```

int matchCount = 0;
for (int i = 0; i < vec.size(); i++) {
    if (choice == 1 && isEven(vec[i])) matchCount++;
    if (choice == 2 && isOdd(vec[i])) matchCount++;
}

cout << "Total elements matching property: " << matchCount << endl;
return 0;
}

```

Solution Summary:

Leverages Standard Template Library collection sequences to handle incoming numeric data. Tracks state evaluation via functional parameter wrappers, accumulating records dynamically on linear traversals.

Practical 13

Write a C++ program that implements a generic function template to sort an array of any data type using Bubble Sort or Selection Sort.

```

#include <iostream>
using namespace std;

template <typename T>
void bubbleSort(T arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

template <typename T>
void displayArray(T arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int iArr[5] = {5, 2, 9, 1, 6};
    float fArr[4] = {3.3, 1.1, 4.4, 2.2};

    bubbleSort(iArr, 5);
    bubbleSort(fArr, 4);

    cout << "Sorted Integer Array: ";
    displayArray(iArr, 5);
}

```

```

    cout << "Sorted Float Array: ";
    displayArray(fArr, 4);

    return 0;
}

```

Solution Summary:

Applies compile-time macro code generation via typename templates. Decouples strict type-binding dependencies, allowing a unified mathematical sorting layout to support variable data elements uniformly.

Practical 14

Design and implement an AI-based Intelligent System using OOP principles for a real-world application (Rule-based Medical Diagnosis System).

```

#include <iostream>
using namespace std;

class MedicalExpertSystem {
    int fever;
    int cough;

public:
    void getSymptoms() {
        cout << "Do you have fever? (1 for Yes, 0 for No): ";
        cin << fever;
        cout << "Do you have cough? (1 for Yes, 0 for No): ";
        cin << cough;
    }

    void diagnose() {
        cout << "\n--- Diagnostic Analysis Result ---" << endl;
        if (fever == 1 && cough == 1) {
            cout << "Recommendation: High probability of Viral Infection. Rest and hydrate." <<
endl;
        } else if (fever == 1 && cough == 0) {
            cout << "Recommendation: Underlying inflammatory response. Monitor temperature." <<
endl;
        } else if (fever == 0 && cough == 1) {
            cout << "Recommendation: Allergic reaction or minor throat irritation suspected." <<
endl;
        } else {
            cout << "Recommendation: System reports normal health parameters." << endl;
        }
    }
};

int main() {
    MedicalExpertSystem aiSystem;
    aiSystem.getSymptoms();
    aiSystem.diagnose();
}

```

```
    return 0;  
}
```

Solution Summary:

Builds a rule-based inference pipeline within class boundaries. Combines encapsulation principles with deterministic boolean validation trees to model primitive artificial evaluation logic.